



Progettazione e Implementazione di un Tool API-Agnostico per la Security Assurance di API REST

Laurea Magistrale in Sicurezza Informatica

Enea Manzi (65935A)

Relatore: Prof. Marco Anisetti

Correlatore: Prof. Claudio Agostino Ardagna

XX Luglio 2026



UNIVERSITÀ
DEGLI STUDI
DI MILANO



Contesto

- Architetture a **microservizi** cloud-native: la superficie di attacco cresce con la **proliferazione di endpoint**
- Il **gateway API** risponde centralizzando l'enforcement delle policy di sicurezza
- La maggior parte degli strumenti di testing verifica la **funzionalità**, non la **sicurezza**
 - sintassi delle richieste, non semantica dei contratti



Gaps

- **G1 Cecità semantica:** scanner e fuzzer operano sulla **sintassi** HTTP, non sulla **semantica** dei contratti
- **G2 Portabilità/profondità:** o **specifico** e *approfondito su un target*, o **superficiale** e *portabile ovunque*
- **G3 Riproducibilità:** l'assenza di **determinismo** preclude l'integrazione nei cicli di **sviluppo continuativo**
- **G4 Oracle problem:** carenza di meccanismi sistematici per stabilire se un comportamento viola una **garanzia di sicurezza**



Obiettivi

- **O1 Tool contract-driven agnostico:** OAS e config.yaml, uniche sorgenti di conoscenza a runtime
- **O2** Conseguenza di O1: applicabile a qualsiasi sistema REST; profondità legata alla **specifica**, non alla **piattaforma**
- **O3 Riproducibilità deterministica:** stessa configurazione, stesso output; indipendente dall'operatore
- **O4** Ogni test ricava il proprio criterio dalla **conoscenza del dominio** e lo applica **deterministicamente**



Agnosticismo applicativo e contract-driven

- **Agnosticismo applicativo:** uniche sorgenti di conoscenza a runtime, `openapi.yaml` e `config.yaml`
 - **config-driven:** nessun riferimento hardcoded al target
 - deployabile su **qualsiasi API REST documentata** senza modifiche
- **Paradigma contract-driven:** **OAS** guida la **costruzione** delle richieste, delimita la **superficie d'attacco**, funge da **oracolo** per alcune valutazioni



Specified oracles: il criterio di valutazione

- Nessun criterio sistematico per giudicare se un **comportamento osservato** è una violazione
- **Specified oracles**: criterio definito *a priori* dalla **conoscenza del dominio** del test e codificato nella sua logica
- Fonti eterogenee, dipendenti dal controllo: *es.* schema OAS, standard tecnici, o costruzioni ad-hoc



Tassonomia e box gradient

- **Do API Discovery**

shadow API, inventario endpoint, ...

- **D1 Authentication**

trasporto e revoca credenziali, TLS, ...

- **D2 Authorization**

RBAC, BOLA, ...

- **D3 Data Integrity**

HMAC, coerenza payload, ...

- **D4 Availability**

rate limiting, circuit breaker, ...

- **D5 Observability**

audit logging, alerting, ...

- **D6 Config & Hardening**

header HTTP, credenziali gateway hardcoded, ...

- **D7 Business Logic**

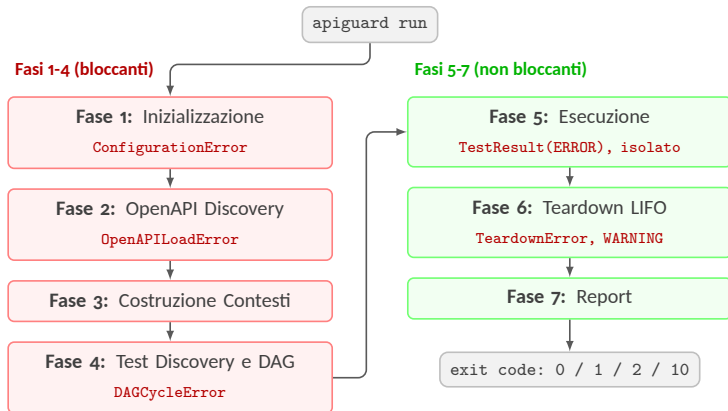
SSRF, race condition, ...

18 test implementati, 14 pianificati, su 8 domini

Ogni **test** dichiara il proprio livello d'accesso: BLACK_BOX → GREY_BOX → WHITE_BOX



Pipeline di esecuzione





Validazione sperimentale

Testbed

- Forgejo 14.0.3 – Kong 3.9 DB-less
- Scelto per le sue proprietà strutturali:
OAS nativa, ruoli multipli

Risultati

- Idempotenza: 2 run indipendenti producono risultati byte-identici
- 18 test, 7 domini: 9 PASS, 7 FAIL, 2 SKIP, 0 ERROR – 98 finding

Una discrepanza reale

Alcuni endpoint pubblici, marcati erroneamente come protetti, rispondono correttamente senza richiedere credenziali.



Conclusioni

Da **osservazione puntuale** a **garanzia** misurabile, riproducibile e **agnostica**

- Tassonomia a **8 domini**, applicabile indipendentemente dall'implementazione
- Metodologia sistematica per gli **specified oracles** nel security testing REST
- Assessment engine deterministico basato su **DAG**



Sviluppi futuri

- **Estensione operativa:** **completamento** di test e connector nei domini parziali, adapter per **gateway cloud managed**
- Piattaforma modulare per **protocolli eterogenei** (gRPC, GraphQL)
- **Formalizzazione** degli oracoli di sicurezza



Progettazione e Implementazione di un Tool API-Agnostico per la Security Assurance di API REST

Grazie per l'ascolto! Qualche domanda?

Enea Manzi: enea.manzi@studenti.unimi.it



Tassonomia e Contesto



Domini di Assurance

Dominio	Titolo	Ambito di verifica
D0	API Discovery	Inventario degli endpoint esposti; rilevazione di Shadow API non documentate nella specifica.
D1	Identity & Authentication	Meccanismi di riconoscimento del chiamante; robustezza del trasporto delle credenziali; ciclo di vita e revoca dei JWT.
D2	Authorization	Logiche di controllo degli accessi; difetti di segregazione orizzontale e verticale (RBAC, BOLA).
D3	Data Integrity	Coerenza strutturale dei payload; implementazione delle firme crittografiche (HMAC).
D4	Availability & Resilience	Difese contro l'esaurimento delle risorse; audit di circuit breaker e soglie di rate limiting.
D5	Observability	Qualità e tempestività della telemetria generata dal sistema.
D6	Configuration & Hardening	Rafforzamento della configurazione del gateway; policy di routing; header di sicurezza HTTP.
D7	Business Logic & Sensitive Flows	Falle semantiche complesse: elusione dei vincoli di processo, SSRF, race condition su operazioni critiche.



Box Gradient

Strategia	Prerequisiti	Razionale
BLACK_BOX	Nessuno (solo accesso di rete)	Controlli perimetrali applicati dal gateway a qualsiasi interlocutore; simulazione di un attaccante esterno senza credenziali.
GREY_BOX	Token per ≥ 2 ruoli distinti	Garanzie che presuppongono stato autenticato con identità di ruolo distinte; inaccessibili senza credenziali valide.
WHITE_BOX	Accesso Admin API del gateway	Configurazione statica verificabile per ispezione diretta tramite piano di controllo del gateway.



Modelli di esposizione gateway

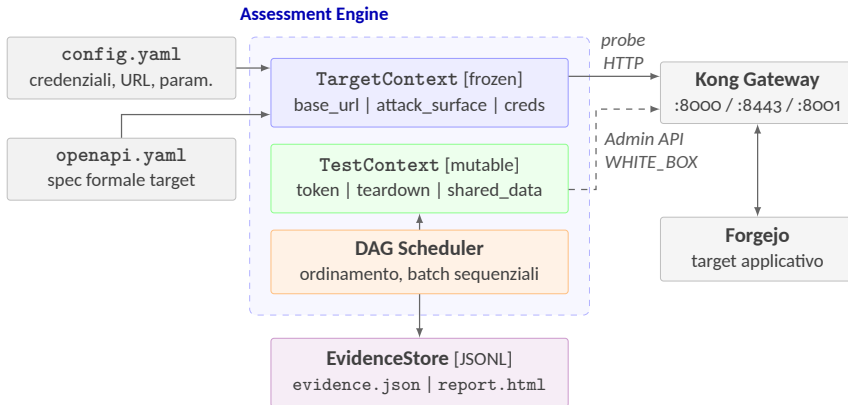
Modello	Traffico presidiato	Enforcement policy	Accesso al piano di configurazione	Verifica WHITE_BOX
Gateway API Dedicato (es. Kong, Apigee)	Nord-sud	Centralizzato (singolo strato)	Admin API diretta	Nativa
Kubernetes Ingress / Gateway API	Nord-sud	Centralizzato (via K8s API)	kubectl / Kubernetes API	Controller-dipendente
Service Mesh (es. Istio)	Est-ovest	Distribuito per sidecar proxy	Control plane (istiod)	N/A (est-ovest)
Gateway Managed Cloud (AWS, Azure, GCP)	Nord-sud	Centralizzato dal provider	SDK / API del cloud provider	Adapter dedicato



Architettura e Stato



Architettura di APIGuard Assurance





Struttura interna del TestContext

TestContext [mutable]: tre canali tipizzati con responsabilità distinte

Canale Token	Canale Teardown [LIFO]	Canale Shared Data
ROLE_ADMIN	push(fn_C)	"1.1.token"
ROLE_USER_A	push(fn_B)	"1.1.repos"
ROLE_USER_B	push(fn_A)	"ext.0.1.hits"
set_token(role, tok) get_token(role) → tok has_token(role) → bool	push_teardown(callable) drain order: fn_A → fn_B → fn_C	set_shared("id.k", v) get_shared("id.k") → v
Fase 5: test autenticati scrivono e leggono	Fase 5: push Fase 6: drain inv.	Fase 5: produttore scrive batch successivi leggono

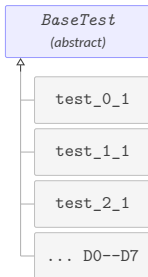


Implementazione e Flusso

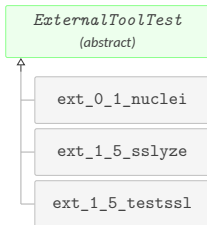


Gerarchia delle classi

Test nativi (DO-D7)

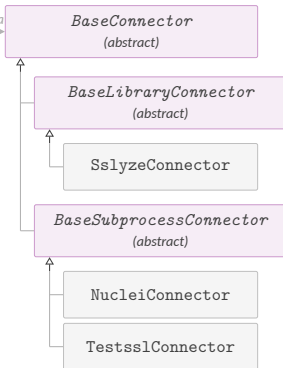


Test esterni



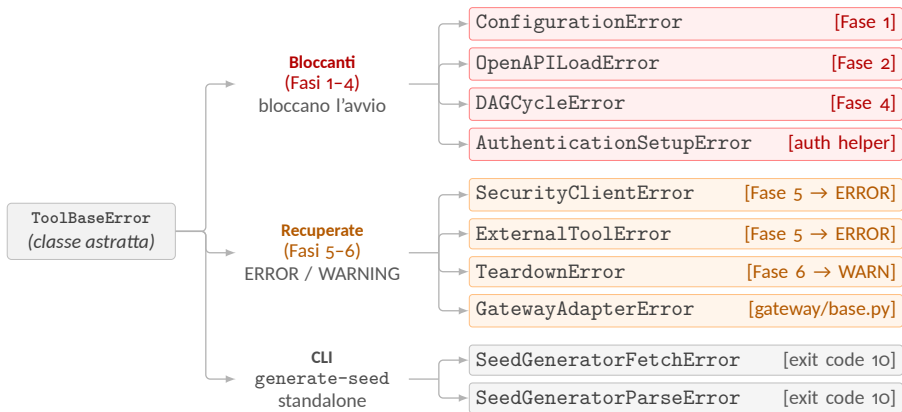
usa

Connettori



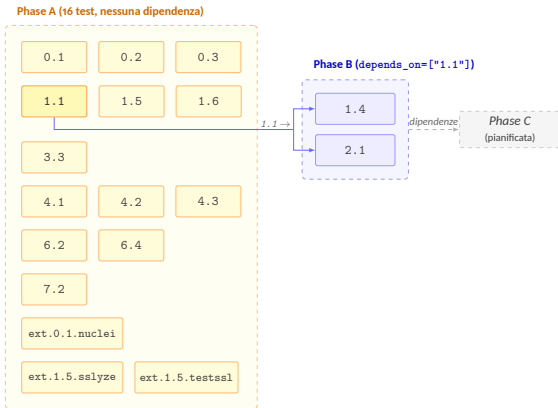


Gerarchia delle eccezioni



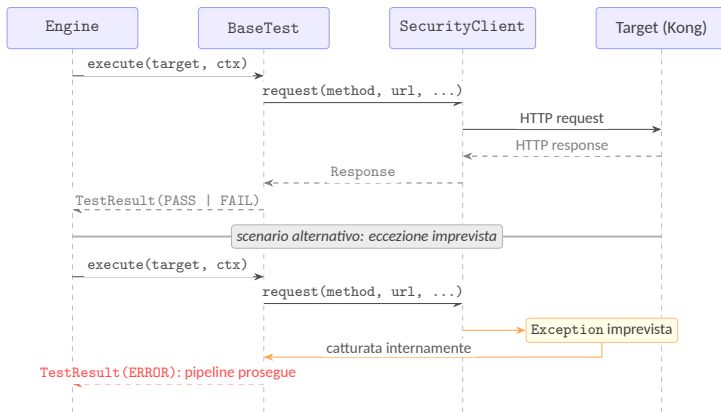


Batch generati dal DAG



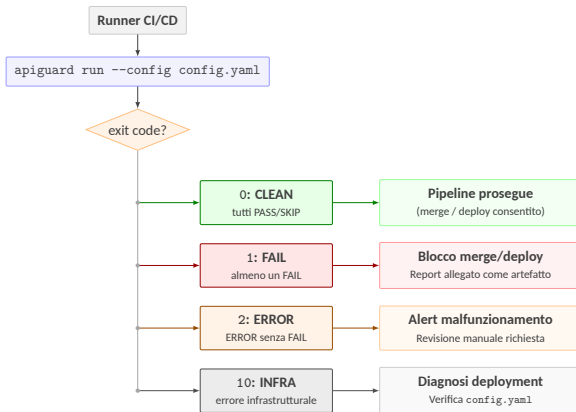


Error isolation in Fase 5





Routing degli exit code

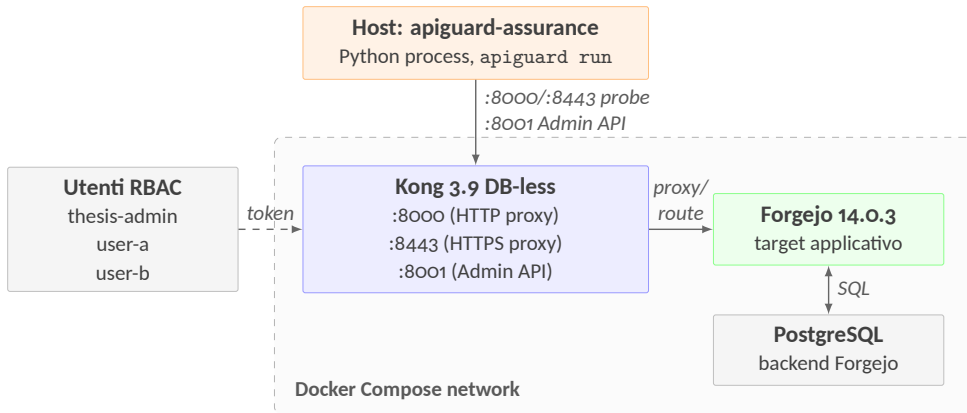




Setup Sperimentale



Topologia del testbed





Versioni del testbed

Componente	Versione	Ruolo nel testbed
apiguard-assurance	0.1.0	Tool sotto validazione
Python	3.12.3	Interprete del tool
Forgejo	14.0.3 (gitea-1.22.0)	Target applicativo: API REST con specifica OpenAPI nativa
PostgreSQL	15-alpine	Backend di persistenza di Forgejo (dipendenza funzionale, non componente autonomo)
Kong	3.9 DB-less	API Gateway: proxy HTTP/HTTPS, instradamento verso Forgejo, Admin API su porta 8001
nuclei	3.8.0	Tool esterno per Shadow API discovery (test ext . 0 . 1)
nuclei-templates	10.4.3	Base di template di scansione versioned: garantisce che lo stesso set di firme venga usato in ogni run
testssl.sh	3.2.3	Analisi TLS black-box (test ext . 1 . 5 . testssl)
sslyze	(libreria Python)	Analisi TLS via libreria Python (test ext . 1 . 5 . sslyze)



Validazione e Risultati



Mappa di copertura





Risultati per test

Test ID	Dominio	Tipo	Status	Finding
0.1	Do: Discovery	Nativo	FAIL	16
0.2	Do: Discovery	Nativo	FAIL	1
0.3	Do: Discovery	Nativo	SKIP	0
1.1	D1: Auth	Nativo	FAIL	74
1.4	D1: Auth	Nativo	PASS	0
1.5	D1: Auth	Nativo	PASS	0
1.6	D1: Auth	Nativo	SKIP	0
2.1	D2: Authorization	Nativo	PASS	0
3.3	D3: Data Integrity	Nativo	PASS	0
4.1	D4: Availability	Nativo	FAIL	2
4.2	D4: Availability	Nativo	PASS	0
4.3	D4: Availability	Nativo	PASS	0
6.2	D6: Hardening	Nativo	PASS	0
6.4	D6: Hardening	Nativo	PASS	0
7.2	D7: Business Logic	Nativo	FAIL	1
ext.0.1.nuclei	Do: Discovery	Esterno	PASS	0
ext.1.5.sslyze	D1: Auth	Esterno	FAIL	1
ext.1.5.testssl	D1: Auth	Esterno	FAIL	3
Totale				98



Analisi statica

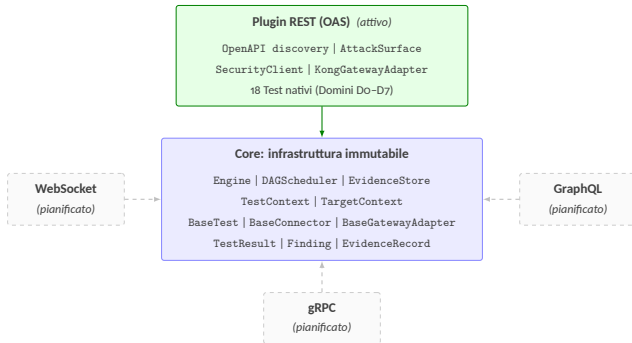
Tool	Esito	Note
ruff	0 errori	Ruleset E/W/F/I/N/UP/B/S/ANN
mypy	0 errori	Modalità strict: no-implicit-optional, disallow-untyped-defs, warn-return-any
bandit	0 finding severity $\geq$ medium	3 Low (B404, S603x2) soppressi con # noqa documentati: invocazione subprocess controllata per design
vulture	0 finding	Confidence $\geq$ 80%; metodi a dispatching dinamico esclusi via whitelist
pip-audit	0 CVE	Nessuna vulnerabilità nota nelle dipendenze dichiarate



Sviluppi Futuri



Verso una piattaforma modulare



freccie continue: modulo attivo frecce tratteggiate: moduli pianificati

Il discovery dinamico via `pkgutil` permette il caricamento trasparente di moduli di terze parti